



State Pattern

1 Mục tiêu bài học

Sau bài học này, sinh viên có thể:

- Hiểu bản chất của State Pattern
- Phân biệt State Pattern với việc dùng if-else hoặc switch
- Thiết kế hệ thống có nhiều trạng thái thay đổi hành vi theo trạng thái
- Áp dụng State Pattern trong Order Management System (OMS)
- Triển khai State Pattern bằng C#

2 Vấn đề

Trong Order Management System, đơn hàng có nhiều trạng thái: New, Paid, Shipped, Delivered, Cancelled. Nếu dùng if-else:

```
if (order.Status == "New")  
    { // xử lý }  
else if (order.Status == "Paid")  
    { // xử lý khác }
```

✗ Vấn đề:

- Code dài và khó bảo trì
- Vi phạm Open/Closed Principle
- Khó mở rộng khi thêm trạng thái mới
- Logic chuyển trạng thái bị phân tán

3 Giải pháp & **4** Cấu trúc

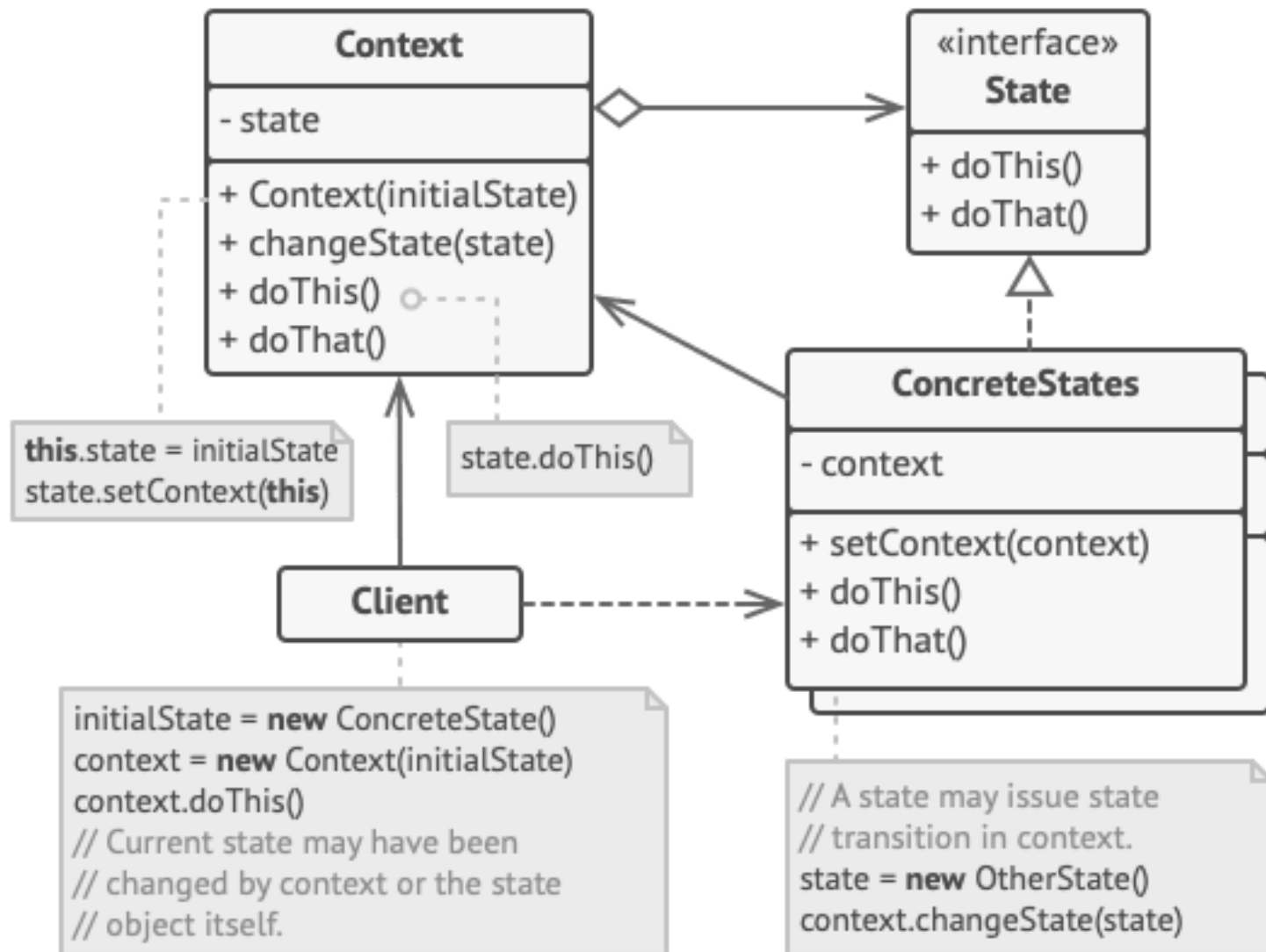
3 Giải pháp: State Pattern

- Ý tưởng: Cho phép một đối tượng thay đổi hành vi khi trạng thái nội tại của nó thay đổi.
- Thay vì if-else, mỗi trạng thái sẽ là một class riêng.

4 Cấu trúc

- IOrderState → State interface
- ConcreteState → Các trạng thái cụ thể
- OrderContext → Context (giữ trạng thái hiện tại)

5 Structure of State Pattern





Ví dụ 1: Quản lý trạng thái đơn hàng (1)



Bước 1: State Interface

```
public interface IOrderState {  
    void Process(OrderContext context);  
    void Cancel(OrderContext context);  
}
```



Bước 2: Context

```
public class OrderContext {  
    public IOrderState State { get; set; }  
    public OrderContext() { State = new NewOrderState(); }  
    public void Process() { State.Process(this); }  
    public void Cancel() { State.Cancel(this); }  
}
```



Ví dụ 1: Concrete States (New & Paid)

◆ New State

```
public class NewOrderState : IOrderState {  
    public void Process(OrderContext context) {  
        Console.WriteLine("Processing payment...");  
        context.State = new PaidOrderState();  
    }  
    public void Cancel(OrderContext context) {  
        Console.WriteLine("Order cancelled.");  
        context.State = new CancelledOrderState();  
    }  
}
```

◆ Paid State

```
public class PaidOrderState : IOrderState {  
    public void Process(OrderContext context) {  
        Console.WriteLine("Shipping order...");  
        context.State = new ShippedOrderState();  
    }  
}
```



Ví dụ 1: Concrete States (Shipped & Delivered)

◆ Shipped State

```
public class ShippedOrderState : IOrderState {  
    public void Process(OrderContext context) {  
        Console.WriteLine("Order delivered.");  
        context.State = new DeliveredOrderState();  
    }  
    public void Cancel(OrderContext context) {  
        Console.WriteLine("Cannot cancel. Order already shipped.");  
    }  
}
```

◆ Delivered State

```
public class DeliveredOrderState : IOrderState {  
    public void Process(OrderContext context) {  
        Console.WriteLine("Order already delivered.");  
    }  
}
```




Ví dụ 2: Thanh toán

```
class Program {  
    static void Main(string[] args) {  
        OrderContext order = new OrderContext();  
        order.Process(); // New → Paid  
        order.Process(); // Paid → Shipped  
        order.Process(); // Shipped → Delivered  
    }  
}
```



Ví dụ 2: Thanh toán đơn hàng (Pending, Completed, Failed)

```
public interface IPaymentState { void Pay(PaymentContext context);  
}
```

6

So sánh với cách dùng if-else

If-Else	State Pattern
Khó mở rộng	Dễ thêm state mới
Vi phạm OCP	Tuân thủ OCP
Logic tập trung	Logic phân tán theo state
Khó test	Dễ test từng state

7 Khi nào nên dùng & Lợi ích

- ✓ Đối tượng có nhiều trạng thái
- ✓ Hành vi thay đổi theo trạng thái
- ✓ Có nhiều điều kiện if-else phức tạp
- ✓ Muốn tuân thủ Open/Closed Principle

☁️ Lợi ích trong Order Management System

- ✓ Quản lý vòng đời đơn hàng rõ ràng
- ✓ Tránh lỗi chuyển trạng thái sai
- ✓ Dễ mở rộng (thêm ReturnedState, OnHoldState...)
- ✓ Tăng tính bảo trì

8 Bài tập thực hành & Kết luận

- Bài 1: Thêm trạng thái ReturnedOrderState.
- Bài 2: Chỉ cho phép hoàn tiền khi trạng thái là PaidOrderState.
- Bài 3: Thêm logging mỗi khi chuyển trạng thái.

Kết luận

- State Pattern giúp:
- Loại bỏ if-else phức tạp
- Tổ chức code rõ ràng theo trạng thái
- Dễ mở rộng
- Phù hợp với hệ thống có workflow như Order Management System